

JAVA CONSOLE APPLICATION

Blood Bank Inventory Donor Matcher

A Java console application verifying donor–recipient blood group compatibility and recording clinical checks.

By: Mohd Imaad (2620040060)

K Sri Akshay (2620090093)

PROJECT RATIONALE

Automating Donor–Recipient Blood Compatibility

Purpose: Menu-driven ABO/Rh blood compatibility checks (CAN / CANNOT donate).

Safety & Validation: Checks donor age (18–65) and recipient age (0–120) to prevent fatal mismatches.

Audit History: Stores each check in memory for easy review.

Technology: Core Java—Scanner, loops, if-else, switch, methods, arrays, and recursion.



PRE-CHECK AGE VALIDATION

Donor eligibility: ages 18–65 · Recipient limits: ages 0–120 · Applied prior to cross-matching.

Existing Manual System & Operational Vulnerabilities

MANUAL REFERENCE

Manual Paper Charts

Compatibility verification relies on printed reference charts and staff memory.

HUMAN FACTOR

Human Error Risk

Manual evaluation is slow, fatigue-prone, and lacks automated fail-safes during emergencies.

ELIGIBILITY

No Eligibility Gates

Donor age and health restrictions are not systematically enforced before testing.

RECORDS

Fragmented Record Keeping

Physical paper registers make audit trails and historical lookups slow and unreliable.

Proposed Java Architecture & System Flow

Core program flow

- **Menu-driven console:** Check compatibility, View past checks, or Exit.
- **Age validation:** Re-entry loops enforce donor ages 18–65 and recipient ages 0–120.
- **Donor group logic:** An if-else chain shows whom each donor group can donate to.
- **Recipient logic:** A switch statement sets the compatible result.

Results and repeat checks

- **Stored results:** A String array holds up to 100 records and displays them on request.
- **Recursion option:** The user can check another pair without returning to the menu.
- **Overloaded printRecord():** CANNOT results include a detailed diagnostic header.

Course Outcome (CO) Attainments

Course Outcome (CO)	Outcome Description	Implementation in Project
CO1	Handle user input and validate clinical data	Interactive Scanner input, Datatypes, Operators
CO2	Apply basic Java syntax, data types, and control statements	Variables, if-else decision chains, switch cases, do-while menu, and validation loops
CO3	Used modular methods and arrays to structure programs	checkCompatibility() core logic, printRecord() method overloading, and records[] audit array

System Execution & Console Workflow

Three simple figures show the program from menu selection to compatibility results and past checks.

FIGURE 1

Main menu

Blood Bank Inventory Donor System

```
PS C:\Users\Mohd Imaad> & 'C:\Program Files\jdk-26.0.2\bin\java.exe' -jar 'C:\Program Files\jdk-26.0.2\bin\ory_Donor_Matcher6'

===== Blood Bank Inventory Donor System =====
1. Check donor-recipient compatibility
2. View all past checks
3. Exit
Enter your choice: █
```

FIGURE 2

Compatibility result

Donor-recipient

```
=====
Donor age is valid.
Recipient age is valid.
=====
Donor blood group is A+. This blood group can donate to A+ and AB+.
---- Detailed Record ----
test (A+) CANNOT donate to test2 (B-)

Do you want to check another donor (Y/N): █
```

FIGURE 3

Viewing past checks

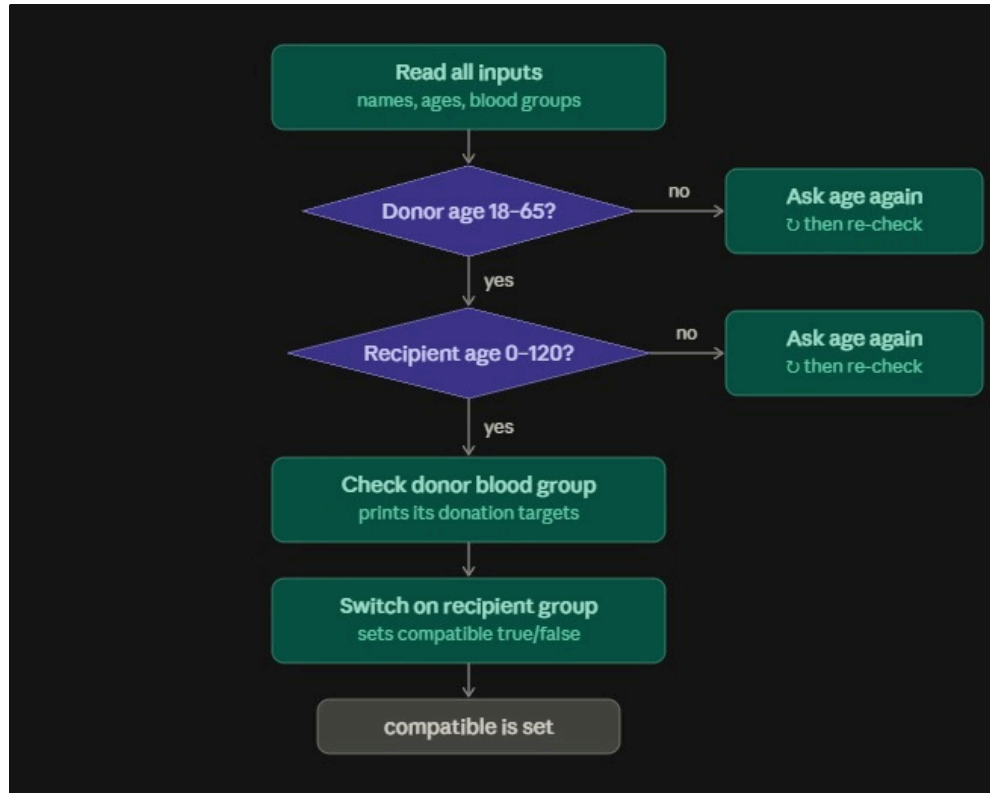
Record history

```
Do you want to check another donor (Y/N): N
Thank you for using the Blood Bank Inventory Donor System

===== Blood Bank Inventory Donor System =====
1. Check donor-recipient compatibility
2. View all past checks
3. Exit
Enter your choice: 2

----- Past Checks -----
test (A+) CANNOT donate to test2 (B-)
```

Flowchart for the Program



Part 1: Input Acquisition, Age Validation Loops & Group Logic

- **Age limits:** invalid input loops back for re-entry.
- **Compatible check:** matching groups move to evaluation.

Part 2: Compatibility Evaluation, Record Storage & Recursive Calling

- **Storage capacity:** save only when a slot is available.
- **Recursion loop:** continue until the workflow completes.

